

LangChain Foundation Guide

This guide walks you through the Foundation phase of LangChain for JavaScript/TypeScript, assuming no prior knowledge. We'll cover installation, models, prompt creation, and output parsing in LangChain. Each topic includes Concept and Theory, a Practical Lab with step-by-step instructions and code examples, and Knowledge Checks. By the end of this phase, you'll be ready to build basic LangChain applications.

Unit 1: Setup & Environment

Concept and Theory

LangChain is a framework for building applications with large language models (LLMs). In JavaScript/TypeScript, LangChain is split into packages. The core package `@langchain/core` provides interfaces and utilities, while integration packages (like `@langchain/openai`) provide model-specific classes. For example, `@langchain/openai` includes classes for OpenAI's models (Chat and non-Chat).

Before coding, we need to manage credentials. LangChain (and the OpenAI SDK) use environment variables to store API keys securely. Typically you set an `OPENAI_API_KEY` environment variable (e.g. in a `.env` file or your shell) so your code can authenticate with OpenAI.

LangChain also introduces the LangChain Expression Language (LCEL): a declarative way to compose chains of operations. With LCEL, you can "pipe" components together. For example, a prompt and a model can be chained as `prompt | model` and then invoked together. This makes chaining operations (like `prompt → model → parser`) very straightforward.

Finally, become familiar with the LangChain docs. The official documentation is on docs.langchain.com. It is organized by component (LLMs, chat models, embeddings, etc.). You can browse the API Reference and integration guides. For example, the OpenAI Chat model guide shows installation and usage.

Practical Lab

Follow these steps to set up a basic Node project with LangChain and make a simple model call.

1. **Initialize a Node project.** Open a terminal and create a new directory:

```
mkdir langchain-fundamentals
cd langchain-fundamentals
npm init -y
```

This creates a `package.json` file (with default settings) so we can install packages.

2. **Install LangChain packages.** LangChain's OpenAI integration lives in the `@langchain/openai` package, and core functionality is in `@langchain/core`. Run:

```
npm install @langchain/core @langchain/openai dotenv
```

- `@langchain/core` includes core classes and interfaces.
- `@langchain/openai` includes `ChatOpenAI`, `OpenAI` (for non-chat LLMs),

and embeddings.

- `dotenv` is a helper to load environment variables from a `.env` file.

3. **Set up credentials.** Register at OpenAI and obtain an API key. Create a file named `.env` in your project root with the content:

```
OPENAI_API_KEY=your-openai-key-here
```

This stores your API key. LangChain examples use the `OPENAI_API_KEY` environment variable. Our code will use `dotenv.config()` to load this key.

4. **Write a simple script.** Create `index.js` (or `index.ts`). In it, load `dotenv` and import a model. For example, use the chat model `ChatOpenAI` for a translation:

```
import dotenv from "dotenv";
dotenv.config(); // Load OPENAI_API_KEY from .env

import { ChatOpenAI } from "@langchain/openai";

async function main() {
  // Instantiate a chat model (GPT)
  const model = new ChatOpenAI({
    model: "gpt-3.5-turbo",
    temperature: 0, // low randomness for deterministic output
  });

  // Prepare a conversation prompt: system + user
  const messages = [
    { role: "system", content: "You are a helpful assistant that translates English to French." },
    { role: "user", content: "I love programming." }
  ];

  // Call the model
  const aiMessage = await model.invoke(messages);
  console.log("Model response:", aiMessage.content);
}

main().catch(console.error);
```

- **Line by line:** We import and configure `dotenv`. We create a `ChatOpenAI` instance with `gpt-3.5-turbo` and a temperature of 0 for reproducible output. We form a list of message objects (system and user roles). We then call `model.invoke(messages)`. This returns an object whose `.content` is the assistant's reply. The console should print something like:

```
Model response: J'adore la programmation.
```

(Expected output: French translation of "I love programming.")

5. **Run the script.** In the terminal, run `node index.js`. You should see the translated sentence printed. If you see an error about missing API key, double-check `.env`. If successful, the model call worked.
6. **Explore the docs.** Take a moment to open docs.langchain.com. For example, search for "ChatOpenAI integration" or "OpenAI integration". Notice how guides show installation,

instantiation, and invocation steps with examples.

Code and Commands

Below are the key code snippets from the lab with explanations:

```
npm init -y
npm install @langchain/core @langchain/openai dotenv
```

- The first command creates a new Node project (`package.json`). The second installs necessary packages.

```
// index.js
import dotenv from "dotenv";
dotenv.config(); // Load .env file

import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({
  model: "gpt-3.5-turbo",
  temperature: 0,
});

// System and user messages as per Chat API
const messages = [
  { role: "system", content: "You are a helpful assistant that translates English to French." },
  { role: "user", content: "I love programming." }
];

const aiMessage = await model.invoke(messages);
console.log("Model response:", aiMessage.content);
```

- We import `ChatOpenAI` from LangChain's OpenAI package. We create the model with desired settings (e.g. `temperature: 0` to make it less random). We build a `messages` array with roles (`system` and `user`). Calling `invoke(messages)` sends these to the model. The result is an `AIMessage` object whose `.content` is the text answer.

Explanation of Key Lines:

- `dotenv.config()`: Loads environment vars (like `OPENAI_API_KEY`) from `.env` so LangChain can use the API key.
- `new ChatOpenAI({ model: "...", temperature: 0 })`: Instantiates the LLM. The `model` parameter selects the specific OpenAI model (e.g. `gpt-3.5-turbo`), and `temperature` controls randomness (0 for deterministic).
- `model.invoke(messages)`: Sends the conversation to the model. Since this is a chat model, we pass an array of `{ role, content }` objects.
- `aiMessage.content`: The assistant's reply text. If the call succeeds, `.content` has the answer.

Verifying Each Step:

- After `npm install`, ensure `node_modules` exists and no errors in console.

- After creating `.env`, run `console.log(process.env.OPENAI_API_KEY)` in code to check it loads.
- After the `invoke` call, the program should not throw errors. It should print a reasonable reply (e.g. the French sentence). If the output looks like garbage or an error, check your prompt format and API key.

Knowledge Check

Summary: We set up a Node project, installed `@langchain/openai` and `@langchain/core`, and managed credentials via an `OPENAI_API_KEY` environment variable. We wrote a simple chat invocation using `ChatOpenAI.invoke()` and saw the model's output.

Key Takeaways:

- LangChain JS uses packages for core and providers. The OpenAI chat model is in `@langchain/openai`.
- Always store API keys in environment variables, not in code.
- `ChatOpenAI` expects an array of message objects for `invoke()`.
- `temperature` and other model parameters control output; e.g. `temperature: 0` yields more deterministic text.

Practice Questions:

- What environment variable must be set to use OpenAI in LangChain JS? (Answer: `OPENAI_API_KEY`.)
- What npm packages did we install for LangChain chat models? (Answer: `@langchain/core` and `@langchain/openai`.)
- In a chat model invocation, what data structure represents the conversation history? (Answer: an array of message objects with `role` and `content`.)

Exercises:

- Create a script that uses `OpenAI` (non-chat) instead of `ChatOpenAI` to complete a prompt. Compare the output format.
- Modify the `temperature` parameter to 0.9. Run the translation multiple times; observe how the responses vary.

Mini Project: Build a simple Node.js program that asks the user for an English sentence (e.g. via `prompt-sync` or command line argument), calls the LangChain chat model to translate it to another language, and logs the result. Ensure you handle missing API key errors.

Unit 2: Models

Concept and Theory

LangChain distinguishes LLM (text-completion) models from Chat models. Legacy LLMs (like GPT-3 text completions) take a single string prompt and return a string response. Chat models (like

GPT-3.5/GPT-4) take a list of messages (with roles `system`, `user`, `assistant`) and return a message object. In LangChain, chat models are prefixed with "Chat" (e.g. `ChatOpenAI`), and if you use an LLM model, the return type is a plain string. As the docs explain, "if the return type of your invocation is a string, ensure you are using a chat model." In other words, use `ChatOpenAI` for list-of-messages interactions; use `OpenAI` (text) for string prompts.

Embedding models convert text into high-dimensional vectors (arrays of numbers) that capture semantic meaning. These embeddings allow you to compare texts by similarity. For example, "cat" and "feline" will have embedding vectors that are close together. LangChain provides an interface to many embedding providers (OpenAI, Cohere, etc.) and standard methods `embedDocuments` and `embedQuery`.

LangChain supports multiple model providers. OpenAI is one, but others include Anthropic, Mistral, Google, etc. Each provider has its own integration package (e.g. `@langchain/anthropic`, `@langchain/google-genai`). The API and behavior might vary slightly (e.g. Anthropic's Claude vs OpenAI's GPT). You typically install and instantiate the class corresponding to the provider you choose.

Structured output refers to getting JSON or typed data rather than free text. Many models (especially OpenAI's latest) support function-calling or schema output, but LangChain can enforce structure. The `withStructuredOutput()` method on a chat model wraps it so that it returns an object matching a Zod schema. Under the hood, LangChain injects format instructions telling the model how to format its output. For models that lack native JSON output, LangChain offers output parsers (covered in Unit 4) to convert text into structured data.

Finally, model parameters control generation behavior. Key parameters include:

- **Temperature:** controls randomness (0 = deterministic, higher = more creative).
- **Max tokens:** limits output length (longer max = longer responses).
- **Stop sequences:** phrases at which generation should end. You pass these via parameters to the model.

Understanding these lets you tweak the model's responses.

Practical Lab

This lab explores calling different types of models, generating embeddings, and enforcing structured output.

1. **Call an LLM vs a Chat model.** Using the same prompt "Hello, how are you?", compare outputs:

```
import { OpenAI } from "@langchain/openai";
import { ChatOpenAI } from "@langchain/openai";

const textModel = new OpenAI({ model: "gpt-3.5-turbo-instruct",
  temperature: 0 });
const chatModel = new ChatOpenAI({ model: "gpt-3.5-turbo", temperature:
  0 });

const llmResponse = await textModel.invoke("Hello, how are you?");
console.log("LLM response (string):", llmResponse);
```

```
const chatResponse = await chatModel.invoke([
  { role: "system", content: "You are a friendly assistant." },
  { role: "user", content: "Hello, how are you?" }
]);
console.log("Chat response (AIMessage):", chatResponse.content);
```

- **Explanation:** `OpenAI.invoke()` takes a string and returns a string or completion result. `ChatOpenAI.invoke()` takes an array of messages and returns an `AIMessage` object with `.content`. You should see that both reply in a friendly way, but note the return types differ.

2. Generate embeddings. Create some sample sentences and embed them:

```
import { OpenAIEmbeddings } from "@langchain/openai";

const sentences = [
  "The cat sat on the mat.",
  "A feline was sitting on a rug.",
  "The weather is sunny today."
];

const embedder = new OpenAIEmbeddings();
const vectors = await embedder.embedDocuments(sentences);
console.log("Embedding vectors for each sentence:");
console.log(vectors);
```

- **Explanation:** We use `OpenAIEmbeddings`, which calls OpenAI's embedding API to convert each sentence into a vector. The output `vectors` is an array of arrays (one per sentence). You should see three numeric vectors of equal length (often 1536 dimensions). Compare the first two vectors (about cats) – they should be more similar to each other than to the weather sentence. You can compute cosine similarity to verify if desired.

3. Enforce structured output with `.withStructuredOutput()`. Define a simple schema and use it on the chat model:

```
import { z } from "zod";

const traitSchema = z.object({
  traits: z.array(z.string()).describe("List of traits mentioned"),
});

const model = new ChatOpenAI({ model: "gpt-3.5-turbo" })
  .withStructuredOutput(traitSchema, { name: "extract_traits", strict: true });

const result = await model.invoke([
  { role: "system", content: "Extract key traits from the user statement." },
  { role: "user", content: `I am 6'5" tall and love fruit.` }
]);
console.log("Structured output:", result);
```

- **Explanation:** We define a Zod schema with one field `traits` (an array of strings). Calling `.withStructuredOutput(schema, {strict: true})` wraps the model so it will output data matching that schema. For example, it might return:

```
{ "traits": ["6'5\" tall", "love fruit"] }
```

The `strict: true` option ensures the schema is enforced strictly. If the model tries to output something else, LangChain will throw an error.

4. **Use a JSON Output Parser.** Suppose a model does not natively support structured output. We can pipe a JSON parser:

```
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { JSONOutputParser } from "@langchain/core/output_parsers";

// Build a prompt template instructing JSON structure
const template = ChatPromptTemplate.fromMessages([
  ["system", `Return details about {topic} as JSON with keys:\n{\n"name":\nstring,\n"description": string,\n"useCases": string[]\}\n`],
  ["user", "{topic}"],
]);

const model = new ChatOpenAI({ model: "gpt-3.5-turbo", temperature: 0 });
const jsonParser = new JSONOutputParser();

const chain = template.pipe(model).pipe(jsonParser);
const output = await chain.invoke({ topic: "WebRTC" });
console.log("Parsed JSON output:", output);
```

- **Explanation:** We create a chat prompt that asks the model to output JSON in a given format. The `JSONOutputParser` will parse the text output into an object. For example, it might print something like:

```
{
  "name": "WebRTC",
  "description": "WebRTC is ...",
  "useCases": ["Video calls", "File sharing", "Gaming"]
}
```

The parser strips away any extra text and converts to a JavaScript object.

5. **Handle parse errors.** If the model's output isn't valid JSON, the parser throws an error. We can catch it and retry:

```
try {
  const output = await chain.invoke({ topic: "WebRTC" });
  console.log("Parsed JSON output:", output);
} catch (e) {
  console.error("Parsing failed, retrying with simpler prompt...");
  // Possible fallback: adjust prompt or use a string parser
}
```

- **Explanation:** In practice, a model may sometimes produce invalid JSON. Wrapping `invoke()` in try/catch lets us detect this and apply a fallback (like re-prompting or using a lax parser). Error handling is crucial for robust apps.

Code and Commands

Key code excerpts from the labs above:

```
// 1. Calling LLM vs Chat models
const textModel = new OpenAI({ model: "gpt-3.5-turbo-instruct", temperature: 0 });
```

```
const chatModel = new ChatOpenAI({ model: "gpt-3.5-turbo", temperature: 0 });

const llmResponse = await textModel.invoke("Hello, how are you?");
console.log("LLM response:", llmResponse);

const chatResponse = await chatModel.invoke([
  { role: "system", content: "You are a friendly assistant." },
  { role: "user", content: "Hello, how are you?" }
]);
console.log("Chat response:", chatResponse.content);
```

- **Explanation:** `OpenAI.invoke(string)` returns a plain string completion. `ChatOpenAI.invoke(array)` returns an `AIMessage`. This illustrates the input/output difference between LLM vs Chat models.

```
// 2. Generating embeddings
import { OpenAIEmbeddings } from "@langchain/openai";

const sentences = ["The cat sat on the mat.", "A feline was sitting on a rug.", "The weather is sunny today."];
const embedder = new OpenAIEmbeddings();
const vectors = await embedder.embedDocuments(sentences);
console.log(vectors);
```

- **Explanation:** `embedDocuments` returns an array of vectors (arrays of numbers) for each input sentence. Similar concepts will have nearby vectors.

```
// 3. Structured output with withStructuredOutput()
import { z } from "zod";

const traitSchema = z.object({
  traits: z.array(z.string()).describe("List of traits mentioned"),
});

const structuredModel = new ChatOpenAI({ model: "gpt-3.5-turbo" })
  .withStructuredOutput(traitSchema, { name: "extract_traits", strict: true });

const result = await structuredModel.invoke([
  { role: "system", content: "Extract key traits from the user statement." },
  { role: "user", content: `I am 6'5" tall and love fruit.` }
]);
console.log("Structured output:", result);
```

- **Explanation:** `withStructuredOutput()` wraps the model so it returns data matching the Zod schema. The output should be an object like `{ traits: ["6'5" tall", "love fruit"] }`.

```
// 4. Using JSONOutputParser
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { JSONOutputParser } from "@langchain/core/output_parsers";

const template = ChatPromptTemplate.fromMessages([
  ["system", `Return details about {topic} as JSON with keys:
    {
      "name": string,
      "description": string,
      "useCases": string[]
    }`],
  ["user", "{topic}"],
]);
```



```
const model = new ChatOpenAI({ model: "gpt-3.5-turbo", temperature: 0 });
const jsonParser = new JSONOutputParser();

const chain = template.pipe(model).pipe(jsonParser);
const output = await chain.invoke({ topic: "WebRTC" });
console.log("Parsed JSON output:", output);
```

- **Explanation:** We inject format instructions asking the model to return JSON. Piping to `JSONOutputParser` then parses the text into an object. If the model outputs valid JSON, output will be a JS object. If not, an error occurs.

Knowledge Check

Summary: We distinguished LLM (text-completion) models vs Chat models. Chat models take message lists (with `role`) and return `AIMessage` objects, while LLMs take plain strings. We generated semantic embeddings using `OpenAIEmbeddings` and saw how similar sentences have similar vectors. We also explored forcing structured outputs: the `.withStructuredOutput()` method wraps a model to return data matching a Zod schema, while output parsers like `JSONOutputParser` can parse model text into JSON.

Key Takeaways:

- Use `OpenAI` for plain-text LLMs and `ChatOpenAI` for chat. Always match input type to model type.
- Embedding models map text to numeric vectors (use `embedDocuments` or `embedQuery`).
- `withStructuredOutput(schema)` enforces a Zod schema on the output.
- `JSONOutputParser` forces and parses JSON output from any model.
- Model parameters like `temperature` affect creativity; `maxTokens` limits length.

Practice Questions:

- How would you call a chat model versus a non-chat model in code? Give example invocation for each.
- What does `embedDocuments()` do, and what is an embedding used for?
- Explain what `.withStructuredOutput(schema)` does on a `ChatOpenAI` model.
- Why might we use an `OutputParser` like `JSONOutputParser`?

Exercises:

- Change the Zod schema in the structured output example to capture a different field (e.g. a single string `summary`) and test it on a prompt.
- Try using `embedQuery("machine learning")` vs `embedDocuments(["machine learning", "artificial intelligence"])` and compare results.

- Modify the JSON parser lab: use the `StringOutputParser` instead and see what raw output you get.

Mini Project: Create a small function `answerQuestion(question: string): Promise<string>` that uses LangChain to answer a question with a chat model, then another function `answerStructured(question: string): Promise<any>` that returns a structured JSON (with keys `answer` and `confidence`). Use what you've learned about chat models, prompts, and output parsers. Test with a few example questions.

Unit 3: Prompts

Concept and Theory

A `PromptTemplate` in LangChain is a way to define template text with placeholders. Think of it as a fill-in-the-blank template. For example, a template might be `"Translate the following text: {text}"`. You fill in `{text}` with the actual input when running the chain. This separates the prompt logic from the input data.

LangChain offers `PromptTemplate` for single-turn LLM prompts (where the model is addressed as if continuing text), and `ChatPromptTemplate` for chat-style prompts (multiple messages). A `ChatPromptTemplate` consists of one or more messages with roles (system, human, AI). For instance:

```
import { ChatPromptTemplate } from "@langchain/core/prompts";

const prompt = ChatPromptTemplate.fromMessages([
  ["system", "You are a helpful assistant."],
  ["user", "Tell me a joke about {subject}."]
]);
```

This template has a system message and a user message with a placeholder `{subject}`. You later call it with `prompt.format({ subject: "chickens" })` to produce the full message list.

Few-shot prompting means giving examples in the prompt. With `ChatPromptTemplate`, you can include multiple user/assistant pairs. For example, in a math solver, you might show `"User: 2+3 = 5"` then `"User: 7+8 = 15"`. LangChain also provides `FewShotPromptTemplate` for structured few-shot patterns, but a simple approach is to manually include examples in the messages.

A `MessagesPlaceholder` is a special template piece that you can use in a prompt to inject dynamic conversation history. For chatbots, you often keep track of past exchanges. You can build a `ChatPromptTemplate` with a placeholder like `new MessagesPlaceholder("history")`, and then when formatting you supply the past messages to fill that placeholder.

You can also compose prompts dynamically. LangChain allows partial variables where you pre-fill some parts, or you can `.pipe()` a Prompt into a model. The LangChain docs show how to build a chain like `prompt.pipe(model)` and then invoke it with variables.

In summary, prompt templates let you define the fixed and variable parts of a prompt. You specify what instructions to give the model (system/human/assistant messages) and where the user input or

examples go. The template can then be reused for different inputs.

Practical Lab

Let's build and use some prompt templates step by step.

1. **Basic PromptTemplate.** Create a simple text prompt with a variable. For example, instruct a model to report on a topic:

```
import { PromptTemplate } from "@langchain/core/prompts";

const template = "Provide a summary of the topic: {topic}";
const prompt = PromptTemplate.fromTemplate(template, ["topic"]);

const inputVars = { topic: "LangChain" };
const formatted = prompt.format(inputVars);
console.log("Formatted prompt:", formatted);
```

- **Explanation:** Here `{topic}` is a placeholder. We create a `PromptTemplate` with that placeholder. Calling `format({ topic: "LangChain" })` replaces `{topic}` with `"LangChain"`, giving a complete prompt string. You should see:

```
Formatted prompt: Provide a summary of the topic: LangChain
```

2. **ChatPromptTemplate.** Build a chat-style prompt with system and user roles:

```
import { ChatPromptTemplate } from "@langchain/core/prompts";

const chatPrompt = ChatPromptTemplate.fromMessages([
  ["system", "You are an assistant that explains concepts."],
  ["user", "Explain the concept of {term} in simple terms."]
]);

const formattedMsgs = await chatPrompt.formatMessages({ term: "quantum computing" });
console.log("Formatted messages:", formattedMsgs);
```

- **Explanation:** `ChatPromptTemplate.fromMessages` takes an array where each entry is `[role, text]`. `{term}` is a dynamic variable. `formatMessages` fills in `{term}` and returns a list of message objects. The output will be an array like:

```
[
  { role: "system", content: "You are an assistant that explains concepts." },
  { role: "user", content: "Explain the concept of quantum computing in simple terms." }
]
```

3. **Few-shot example.** Add one example exchange before asking the user's question:

```
const fewShotPrompt = ChatPromptTemplate.fromMessages([
  ["system", "You summarize recipes."],
  ["user", "Recipe: Bread = Flour + Water + Yeast."],
  ["assistant", "This recipe combines flour, water, and yeast to make bread dough."],
  ["user", "Recipe: Cake = Flour + Sugar + Eggs + Butter."],
  ["assistant", "This recipe combines flour, sugar, eggs, and butter to make cake batter."],
]);
```

```
["user", "Recipe: Omelette = Eggs + Milk + Salt."]  
]);  
  
const resultMsgs = await fewShotPrompt.formatMessages({});  
console.log("Few-shot prompt messages:", resultMsgs);
```

- **Explanation:** We included two example user-assistant pairs (about bread and cake) before the actual question (omelette). Notice we didn't use placeholders here; this prompt is fixed. When passed to a model, it guides the style. The output `resultMsgs` will be the array of messages (here the last user question is included at the end). We then pipe this into a model:

```
const model = new ChatOpenAI({ model: "gpt-3.5-turbo", temperature:  
0.2 });  
const answer = await model.invoke(resultMsgs);  
console.log("Model answer:", answer.content);
```

The assistant should respond with a summary of the omelette recipe, similar in style to the examples.

4. **MessagesPlaceholder (history injection).** Suppose we have conversation history in an array `history`. We can use a placeholder in the template:

```
import { MessagesPlaceholder } from "@langchain/core/prompts";  
  
const historyPlaceholder = new MessagesPlaceholder("history");  
const historyPrompt = ChatPromptTemplate.fromMessages([  
  ["system", "You are a chatbot."],  
  [historyPlaceholder, ""], // placeholder will be replaced by past  
  messages  
  ["user", "{new_question}"]  
]);  
  
const pastConvo = [  
  { role: "user", content: "Hi, who are you?" },  
  { role: "assistant", content: "I am a chatbot that helps answer your  
  questions." }  
];  
  
const msgs = await historyPrompt.formatMessages({ history: pastConvo,  
new_question: "What can you do?" });  
console.log("Messages with history:", msgs);
```

- **Explanation:** `MessagesPlaceholder("history")` indicates where to insert previous messages. When formatting, we supply an array of past messages as `history`, and it gets spliced into the conversation. The final `msgs` array will have the system message, then the two history messages, then the new user message. A model invoked on `msgs` will see the full context.

5. **Dynamic prompt composition.** We can also chain prompt to model using pipes:

```
// Equivalent to:  
// ChatPromptTemplate.fromMessages([...]).pipe(model).invoke({ ... })  
  
const chain = ChatPromptTemplate.fromMessages([  
  ["system", "You are a helpful assistant."],  
  ["user", "{question}"]  
]).pipe(new ChatOpenAI({ model: "gpt-3.5-turbo", temperature: 0 }));
```

```
const answer = await chain.invoke({ question: "What is LangChain?" });
console.log("Chain response:", answer.content);
```

- **Explanation:** Here we used the LCEL style with `|` or `.pipe()`. We created a chat prompt template and immediately piped it into the chat model. Calling `chain.invoke({ question: ... })` fills in `{question}` and runs the model. This is functionally similar to formatting the prompt then invoking the model.

Code and Commands

Important code examples:

```
// Basic PromptTemplate example
import { PromptTemplate } from "@langchain/core/prompts";

const prompt = PromptTemplate.fromTemplate("Translate to French: {text}",
["text"]);
const formattedText = prompt.format({ text: "Hello" });
console.log(formattedText); // "Translate to French: Hello"
```

- **Explanation:** We create a template with a `{text}` placeholder. Calling `format({ text: "Hello" })` injects "Hello".

```
// ChatPromptTemplate example
import { ChatPromptTemplate } from "@langchain/core/prompts";

const chatPrompt = ChatPromptTemplate.fromMessages([
  ["system", "You are an assistant."],
  ["user", "What is the capital of {country}?"]
]);

const messages = await chatPrompt.formatMessages({ country: "France" });
console.log(messages);
/* Prints:
[
  {role: "system", content: "You are an assistant."},
  {role: "user", content: "What is the capital of France?"}
]
*/
```

- **Explanation:** `fromMessages` takes an array of `[role, content]`. The `{country}` variable is filled by `formatMessages`.

```
// Few-shot prompt with examples
const fewShotPrompt = ChatPromptTemplate.fromMessages([
  ["system", "Answer math questions."],
  ["user", "What is 2 + 3?"],
  ["assistant", "2 + 3 = 5"],
  ["user", "What is 7 + 8?"],
  ["assistant", "7 + 8 = 15"],
  ["user", "What is 12 + 4?"]
]);

const fewShotMsgs = await fewShotPrompt.formatMessages({});
console.log(fewShotMsgs);
```

- **Explanation:** We included example Q&A pairs before the final question. The model will see those examples as context.

```
// Using MessagesPlaceholder to inject history
import { MessagesPlaceholder } from "@langchain/core/prompts";

const historyTemplate = ChatPromptTemplate.fromMessages([
  ["system", "Chatbot."],
  [new MessagesPlaceholder("history"), ""],
  ["user", "{new_q}"]
]);

const pastMessages = [
  {role: "user", content: "Hello"},
  {role: "assistant", content: "Hi, how can I help?"}
];

const fullConvo = await historyTemplate.formatMessages({ history: pastMessages,
  new_q: "Tell me a joke." });
console.log(fullConvo);
```

- **Explanation:** The `MessagesPlaceholder("history")` is filled by the array `pastMessages`. The result is a merged conversation history with the new query at the end.

Knowledge Check

Summary: We learned to create `PromptTemplate` and `ChatPromptTemplate` instances to programmatically build prompts. `PromptTemplate` is for single-string prompts with variables, while `ChatPromptTemplate` allows multiple messages with roles. We saw how to add few-shot examples and how to inject conversation history with `MessagesPlaceholder`.

Key Takeaways:

- A template uses `{variable}` placeholders which you fill in with `format()` or `formatMessages()`.
- `ChatPromptTemplate.fromMessages([...])` lets you specify system, human, or assistant messages with placeholders.
- Few-shot prompts include example message pairs. They help "prime" the model's response style.
- `MessagesPlaceholder("history")` can be used in chat prompts to insert previous messages dynamically.
- After building a prompt template, you can pipe it into a model (e.g. `prompt.pipe(model)`) and invoke with an input object.

Practice Questions:

- How do you define a placeholder variable in a prompt template? How do you fill it later?
- What's the difference between `PromptTemplate` and `ChatPromptTemplate`?
- Describe what few-shot prompting is and how to do it with `LangChain`.
- Explain how to include past conversation history in a new prompt with `MessagesPlaceholder`.

Exercises:

- Write a `ChatPromptTemplate` that takes a user's name and says goodbye using a system message. Fill it with two different names.
- Build a few-shot prompt for sentiment analysis: show one positive and one negative example, then ask about a new sentence.
- Implement a simple command-line chat loop: keep an array of messages and on each user input, build a prompt using that history and get a response from the model.

Mini Project: Create a small chatbot that maintains conversation context. Each turn, use a `ChatPromptTemplate` with a `MessagesPlaceholder` to include all previous turns. Let the user type inputs (e.g., via the `readline` module) and have the bot respond. Test that it remembers past information (like the user's name once provided).

Unit 4: Output Parsers

Concept and Theory

When LLMs generate text, the output is usually free-form natural language. Output Parsers force or convert this output into a predictable format. LangChain provides several built-in parsers:

- **StringOutputParser:** The simplest parser. It takes the raw model output and returns it as a plain string (typically stripping whitespace). Use this when you just need the text with no extra formatting.
- **JSONOutputParser:** Forces the model to output valid JSON. It injects instructions into the prompt so the model knows to output JSON, then parses the output into an object. This is useful when you need a structured format for things like APIs or database insertion.
- **StructuredOutputParser (schema-based):** You define a schema (using Zod or JSON schema) describing the expected output structure. LangChain will inject the schema as instructions and validate the output against it. This ensures the model's output has exactly the fields and types you want. It is similar to function-calling but works with any model. For example, you can require fields like `{"answer": string, "confidence": number}`. If the model deviates, an error is thrown.
- **CSVOutputParser:** Makes the model output data in CSV table form (rows and columns). Good for tabular data.

Each parser typically has a method to get "format instructions" that you include in your prompt template. For example, `const instructions = jsonParser.getFormatInstructions();` returns a string explaining how the JSON should look. You include these instructions in your system prompt so the model knows the required format.

Using an output parser is essentially the last step in a chain: `Prompt -> Model -> Parser`. After the model runs, the parser processes the text. For chat models, LangChain can also use the `withStructuredOutput()` method to do this in one step (under the hood it uses a `StructuredOutputParser`).

Practical Lab

We will practice using different output parsers.

1. **StringOutputParser.** If you just want the text response, wrap the model in a StringOutputParser:

```
import { StringOutputParser } from "@langchain/core/output_parsers";
import { ChatPromptTemplate } from "@langchain/core/prompts";

const template = ChatPromptTemplate.fromMessages([
  ["system", "You are an assistant that answers questions."],
  ["user", "Translate the word '{word}' to French."]
]);

const model = new ChatOpenAI({ model: "gpt-3.5-turbo", temperature: 0 });
const stringParser = new StringOutputParser();

const chain = template.pipe(model).pipe(stringParser);
const result = await chain.invoke({ word: "Hello" });
console.log("String parser result:", result);
```

- **Explanation:** The StringOutputParser does no special parsing; it effectively returns the model's text. If the model said "Bonjour", you'd get "Bonjour". This is useful when you only care about the text content with no JSON.

2. **JSONOutputParser.** We've seen an example above, but let's do it with format instructions:

```
import { JSONOutputParser } from "@langchain/core/output_parsers";
import { ChatPromptTemplate } from "@langchain/core/prompts";

const jsonParser = new JSONOutputParser();
const formatInst = jsonParser.getFormatInstructions();

const jsonTemplate = ChatPromptTemplate.fromMessages([
  ["system", `You are a helpful assistant. ${formatInst}`],
  ["user", "Give me 2 famous scientists as JSON with fields name and field."]
]);

const jsonChain = jsonTemplate.pipe(new ChatOpenAI({ model: "gpt-3.5-turbo", temperature: 0 }));
const jsonResult = await jsonChain.invoke({});
console.log("JSON parser result:", jsonResult);
```

- **Explanation:** We call `getFormatInstructions()` to get text like "Make sure your output is valid JSON with no extra text." We include this in the system prompt. The user asks for "2 famous scientists". The model should list them in JSON form:

```
[
  { "name": "Albert Einstein", "field": "Physics" },
  { "name": "Marie Curie", "field": "Chemistry" }
]
```

The JSONOutputParser parses this string into a JavaScript array of objects.

3. **StructuredOutputParser (Zod schema).** Suppose we want the answer and a confidence score:

```
import { StructuredOutputParser } from "@langchain/core/output_parsers";
```



```
import { z } from "zod";

const schema = z.object({
  answer: z.string().describe("The answer string."),
  confidence: z.number().describe("A confidence score between 0 and 1.")
});

const parser = StructuredOutputParser.fromZodSchema(schema);
const formatInstr = parser.getFormatInstructions();

const schemaTemplate = ChatPromptTemplate.fromMessages([
  ["system", `Answer the question. ${formatInstr}`],
  ["user", "What color is the sky?"]
]);

const schemaChain = schemaTemplate.pipe(new ChatOpenAI({model: "gpt-3.5-turbo", temperature: 0}));
const schemaResult = await schemaChain.invoke({});
console.log("Structured parser result:", schemaResult);
```

- **Explanation:** We define a Zod schema for the response. `fromZodSchema` creates a parser. We get its format instructions and include them. The model is prompted to answer "What color is the sky?". The parser ensures the output matches the schema, e.g.:

```
{ "answer": "Blue", "confidence": 0.85 }
```

The parser will return a JavaScript object with these fields (or throw an error if the model output is invalid).

4. **Handling parse errors.** To see what happens on error, try giving an output not matching the schema. For example:

```
// Prompt a false answer deliberately
const badTemplate = ChatPromptTemplate.fromMessages([
  ["system", `Answer the question. ${formatInstr}`],
  ["user", "Say 'red' is the color of the sky."]
]);

try {
  const badResult = await badTemplate.pipe(new ChatOpenAI({model: "gpt-3.5-turbo"})).invoke({});
  console.log("Bad result:", badResult);
} catch (err) {
  console.error("Parse error occurred:", err);
}
```

- **Explanation:** The user prompt is factually incorrect ("sky is red"), but more importantly, the model might output a response that doesn't conform to the schema (if it writes an explanation). The `StructuredOutputParser` will throw a validation error, which we catch. This shows the importance of handling errors: you could then retry with different phrasing.

Code and Commands

```
// 1. StringOutputParser example
import { StringOutputParser } from "@langchain/core/output_parsers";

const stringParser = new StringOutputParser();
```

```
const chain1 = ChatPromptTemplate.fromMessages([
  ["system", "You summarize text."],
  ["user", "Summarize: {text}"]
]).pipe(new ChatOpenAI({model: "gpt-3.5-turbo"})).pipe(stringParser);

const summary = await chain1.invoke({ text: "LangChain is a framework for..." });
console.log("Summary:", summary);
```

- **Explanation:** The string parser just returns the model's summary text.

```
// 2. JSONOutputParser with format instructions
import { JSONOutputParser } from "@langchain/core/output_parsers";

const jsonParser = new JSONOutputParser();
const instructions = jsonParser.getFormatInstructions();

const chain2 = ChatPromptTemplate.fromMessages([
  ["system", `You output JSON. ${instructions}`],
  ["user", "List two fruits and their colors as JSON."]
]).pipe(new ChatOpenAI({model: "gpt-3.5-turbo"})).pipe(jsonParser);

const fruits = await chain2.invoke({});
console.log("Fruits JSON:", fruits);
```

- **Explanation:** The system prompt includes format instructions telling the model to output JSON. The JSON parser then returns an object or array.

```
// 3. StructuredOutputParser with Zod schema
import { StructuredOutputParser } from "@langchain/core/output_parsers";
import { z } from "zod";

const PersonSchema = z.object({
  name: z.string(),
  age: z.number()
});

const structuredParser = StructuredOutputParser.fromZodSchema(PersonSchema);
const structInstructions = structuredParser.getFormatInstructions();

const chain3 = ChatPromptTemplate.fromMessages([
  ["system", `Provide a person's info. ${structInstructions}`],
  ["user", "Name: Alice, Age: 30"]
]).pipe(new ChatOpenAI({model: "gpt-3.5-turbo"})).pipe(structuredParser);

const personInfo = await chain3.invoke({});
console.log("Structured output:", personInfo);
```

- **Explanation:** We expect the model to output something like `{"name": "Alice", "age": 30}`. The parser returns a JS object `{ name: "Alice", age: 30 }`.

Knowledge Check

Summary: Output parsers help structure model responses. We saw that the `StringOutputParser` simply returns the raw text, whereas `JSONOutputParser` ensures valid JSON output and parses it. The `StructuredOutputParser` goes further by enforcing a schema (via Zod). We learned to include parser instructions in the prompt using `getFormatInstructions()` to guide the model.

Key Takeaways:

- Always include clear format instructions in your prompt when using an output parser (many parsers provide these instructions to inject).
- `StringOutputParser` = plain text; `JSONOutputParser` = parsed JSON; `StructuredOutputParser` = schema-validated object.
- If a parser fails (output invalid), it will throw an error. Always handle this (try/catch) and possibly retry.
- Structured output is especially useful for building APIs or returning data to other code because it avoids ambiguous parsing of natural language.

Practice Questions:

- What does `JSONOutputParser` do when connected to a model? (Answer: It forces the model to output valid JSON and parses it into an object.)
- How do you get format instructions from a parser? (Answer: Use `parser.getFormatInstructions()` and include the returned text in your prompt.)
- What happens if the model output doesn't match the Zod schema in `StructuredOutputParser`? (Answer: The parser will throw a validation error.)
- When would you use `StringOutputParser`? (Answer: When you only need the raw text with no special format.)

Exercises:

- Try a `CSVOutputParser` by asking the model to list "Name, Age" of three people. Observe the output string.
- Modify one of the structured output examples to make the schema optional (e.g. using `z.optional()`) and see how missing fields are handled.
- Build a chain that uses multiple parsers sequentially (e.g. model → JSON parser → structured parser).

Mini Project: Combine everything: create a function that takes a user prompt, sends it through a `ChatOpenAI` model and a suitable output parser, and returns the parsed result. The function should handle errors (e.g. parse errors) and log any issues. For example, make a "safe question-answering" function that ensures the answer is returned as JSON with fields `{ answer, sourceUrl }`.

Final Project (Foundation Integration)

Project: Build a basic QA Bot that uses `LangChain (JS)` to answer questions with structured output. The bot should:

- Read user input from the console.
- Maintain a short conversation history (e.g. last 3 messages).

- Use a `ChatPromptTemplate` to form the prompt, including a `MessagesPlaceholder` for history.
- Always instruct the model to output JSON in this format: `{ "answer": <string>, "source": <string> }`.
- Use either `withStructuredOutput()` or `JSONOutputParser` with a Zod schema to parse the response.
- Handle errors if the model output is invalid JSON: in that case, ask the model again (or reply with an error message).
- After parsing, print only the answer to the console.

This will combine environment setup, model invocation, prompts with history, and output parsing in one app. You can start from the code structure in this guide and iteratively build up to the final working bot.

Good luck! With these foundations, you have all the tools to start building powerful LangChain applications.